

Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 02. Patrones de diseño estructurales

Tema:

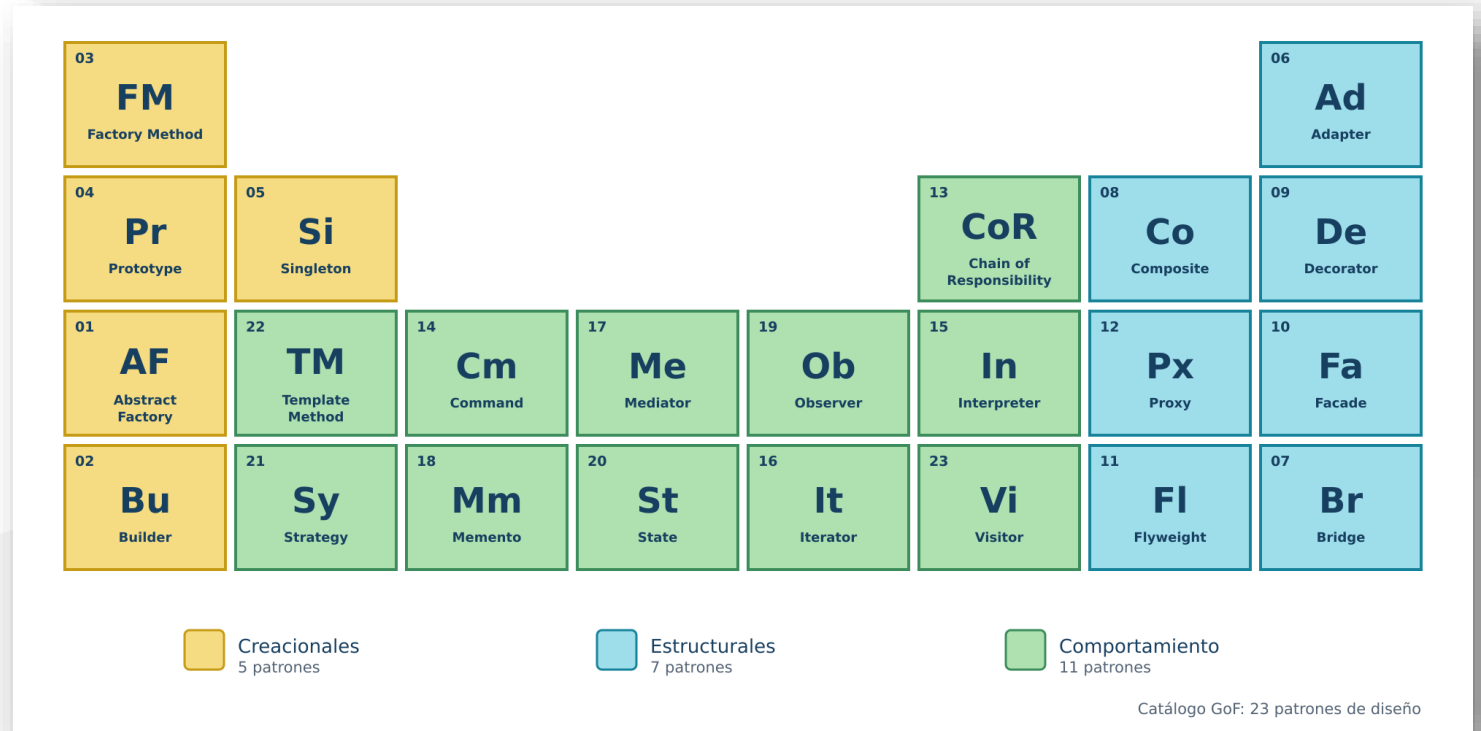
Patrones estructurales

- *Patrones estructurales*
- Composite
- Adapter
- Facade

Introducción a los patrones estructurales

Clasificación de los patrones GoF por propósito

- **Patrones creacionales (5):** Abstraen aspectos del proceso de creación de objetos y reducen la dependencia respecto de clases concretas.
- **Patrones estructurales (7):** Describen cómo componer clases y objetos para formar estructuras mayores manteniendo flexibilidad y extensibilidad.
- **Patrones de comportamiento (11):** Organizan la comunicación, colaboración y distribución de responsabilidades entre objetos.



Patrones estructurales

Introducción

- Los patrones estructurales describen cómo organizar y relacionar clases y objetos para construir estructuras de software más flexibles y extensibles.
- Se utilizan cuando el problema no está en crear los objetos, sino en cómo deben relacionarse, componerse o colaborar.
- Los patrones estructurales de clases utilizan principalmente herencia para relacionar interfaces o implementaciones.
- Los patrones estructurales de objetos utilizan principalmente composición para construir estructuras y distribuir responsabilidades entre objetos.
- Su aplicación puede reducir acoplamiento y facilitar la evolución del diseño, pero también puede introducir nuevas abstracciones e indirecciones.

Patrones estructurales

Catálogo

- Composite: Compone objetos en estructuras de árbol y permite tratar de manera uniforme objetos individuales y composiciones. (Archivos y carpetas)
- Adapter: Permite que objetos con interfaces incompatibles colaboren mediante una interfaz adaptada. (Adaptador de enchufe)
- Bridge: Separa una abstracción de su implementación para que ambas puedan variar independientemente. (Figuras y colores)
- Facade: Proporciona una interfaz simplificada y unificada para acceder a un subsistema complejo. (Reserva de vuelos)
- Decorator: Permite añadir responsabilidades a un objeto dinámicamente sin modificar su clase. (Café con extras)
- Flyweight: Comparte estado común entre múltiples objetos para reducir el consumo de memoria. (Árboles de bosque)
- Proxy: Proporciona un sustituto de otro objeto para controlar o gestionar su acceso. (Acceso a documento).

Tema:

Composite

- Patrones estructurales
- *Composite*
- Adapter
- Facade

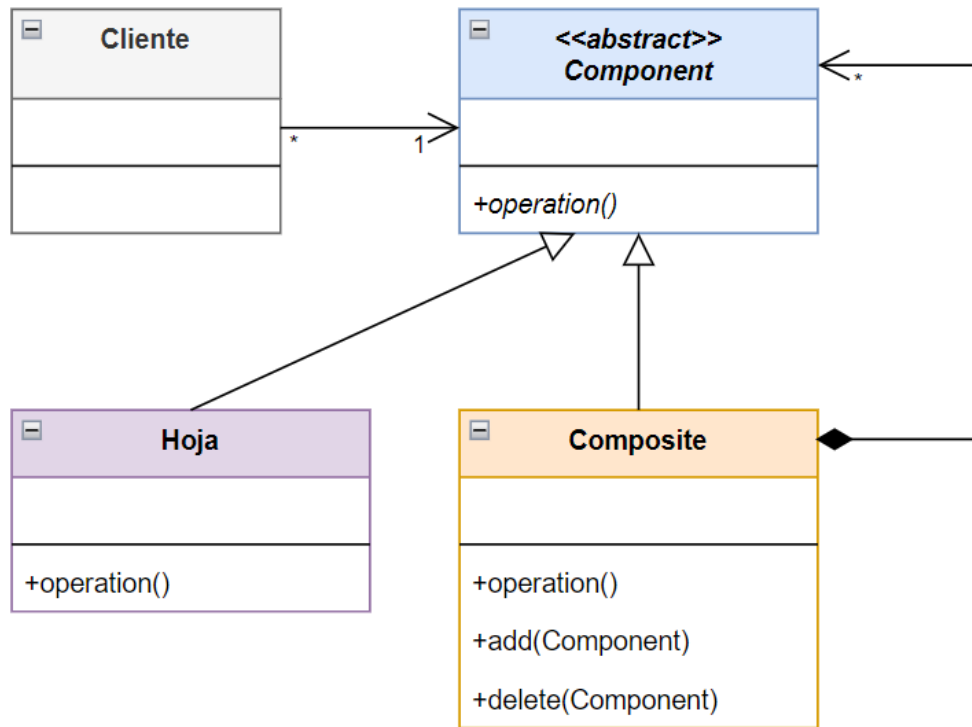
Composite

Composite

- Propósito
 - Compone objetos en estructuras de árbol para representar jerarquías parte-todo.
 - Permite que los clientes traten de manera uniforme tanto a los objetos individuales como a las composiciones de objetos.
- Problema / Motivación
 - Supongamos que deseamos modelar las figuras de un editor gráfico.
 - Existen elementos individuales como Línea, Círculo y Texto, pero también Dibujos formados por varios elementos.
 - Un Dibujo puede contener figuras individuales e incluso otros Dibujos, formando una estructura jerárquica.
 - Todos los elementos deben permitir operaciones comunes, por ejemplo dibujarse y calcular su tamaño en bytes.
 - El problema aparece cuando el cliente debe distinguir continuamente entre una figura individual y una composición para realizar las mismas operaciones.

Composite

Estructura



• Participantes

- **Component**

Define la interfaz común para los objetos de la composición. Puede declarar operaciones comunes y operaciones para gestionar componentes hijos.

- **Hoja:**

Representa los objetos individuales de la composición. No contiene hijos e implementa directamente las operaciones definidas por **Component**.

- **Composite:**

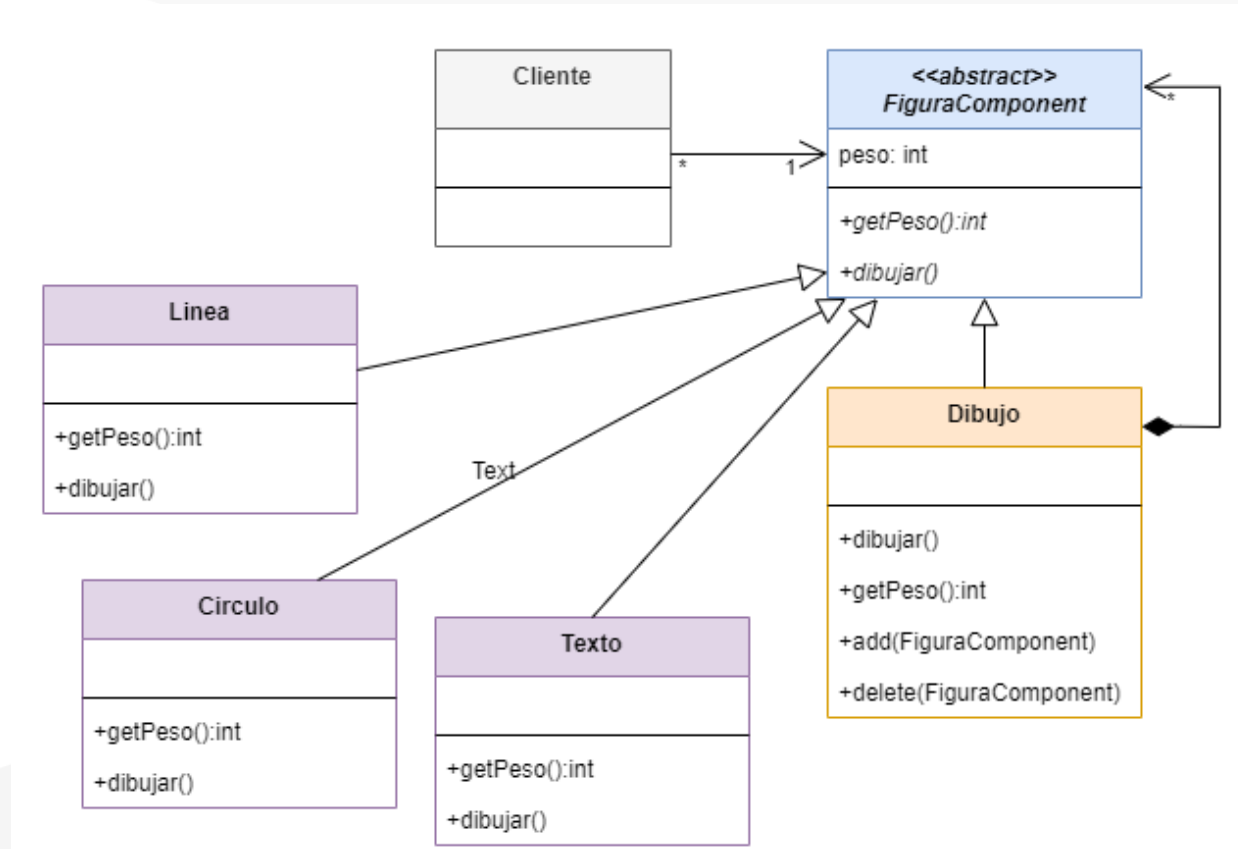
Representa componentes que pueden contener otros **Component**. Almacena y gestiona sus hijos e implementa las operaciones comunes, normalmente delegándolas de forma recursiva.

• Colaboraciones

- El Cliente trabaja con los objetos mediante la abstracción **Component**.
- Si el receptor es una Hoja, ejecuta directamente la operación.
- Si el receptor es un **Composite**, normalmente propaga la operación a sus componentes hijos.

Composite

Solución



<https://github.com/mortizo/pds-unidad02-ooms/tree/master/pds-unidad02-ooms/src/main/java/p69/composite>

Composite

Composite

- Aplicabilidad
 - Cuando se necesita representar jerarquías parte–todo mediante estructuras de objetos.
 - Cuando existen elementos individuales y elementos compuestos que deben responder a operaciones comunes.
 - Cuando una composición puede contener otros elementos e incluso otras composiciones de forma recursiva.
 - Cuando se desea que el cliente trate de manera uniforme tanto a los objetos individuales como a las composiciones.

- Composite es útil cuando necesitamos tratar de la misma manera la parte y el todo.

Composite

Actividad en clase: ¿Composite?

Caso 1 — Gestión de procesos

Una Universidad administra procesos, procedimientos y políticas.

Un proceso puede contener procedimientos y políticas, pero también otros procesos que funcionan como subprocesos.

¿Cómo diseñarían una estructura que permita trabajar de manera uniforme con los elementos individuales y con los procesos que contienen otros elementos?

Caso 2 — Gestión de tareas

Un sistema administra tareas individuales y tareas compuestas.

Una tarea compuesta puede contener varias tareas individuales e incluso otras tareas compuestas.

¿Cómo diseñarían la estructura para ejecutar operaciones sobre una tarea individual o sobre un conjunto completo de tareas de la misma manera?

Caso 3 — Sistema de archivos

Un sistema debe representar archivos y carpetas.

Las carpetas pueden contener archivos y otras carpetas, formando estructuras de varios niveles.

¿Cómo diseñarían el sistema para calcular el tamaño o mostrar el contenido sin que el cliente tenga que distinguir entre archivos y carpetas?

Caso 4 — Menús de aplicación

Una aplicación dispone de opciones individuales y menús.

Un menú puede contener opciones y también otros submenús.

¿Cómo diseñarían la estructura para mostrar o habilitar sus elementos utilizando una operación común independientemente de si se trata de una opción o de un menú?

Tema:

Adapter

- Patrones estructurales
- Composite
- *Adapter*
- Facade

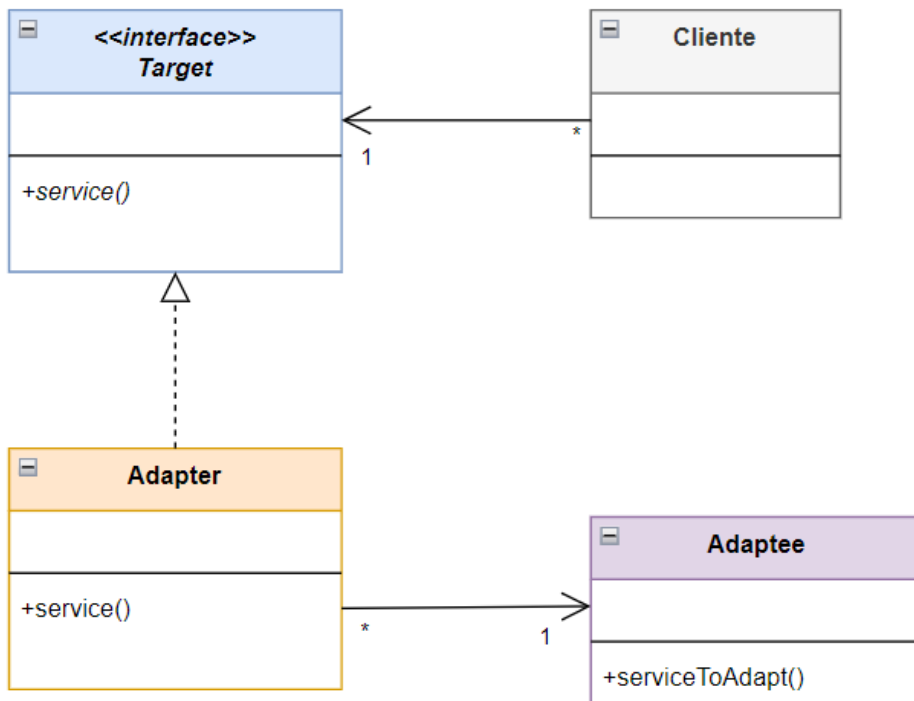
Adapter

Adapter

- Propósito
 - Convierte la interfaz de una clase existente en otra interfaz que es la que espera el cliente.
 - Permite que objetos con interfaces incompatibles puedan colaborar sin modificar necesariamente sus implementaciones existentes.
- Problema / Motivación
 - Una universidad necesita consultar en diferentes bancos si las cuotas de sus estudiantes han sido pagadas.
 - Cada banco proporciona un módulo de integración diferente, con operaciones, parámetros o formatos propios.
 - El sistema interno de la universidad necesita realizar siempre la misma operación: consultar si una determinada cuota está pagada.
 - No resulta conveniente modificar el sistema interno cada vez que se incorpora un banco ni modificar los módulos proporcionados por terceros.

Adapter

Estructura



- Participantes

- Target**

Define la interfaz que espera y utiliza el Cliente.

- Adaptee:**

Representa la clase o componente existente cuya interfaz es incompatible con la requerida por el Cliente.

- Adapter:**

Implementa la interfaz Target y traduce las solicitudes del Cliente hacia las operaciones disponibles en Adaptee.

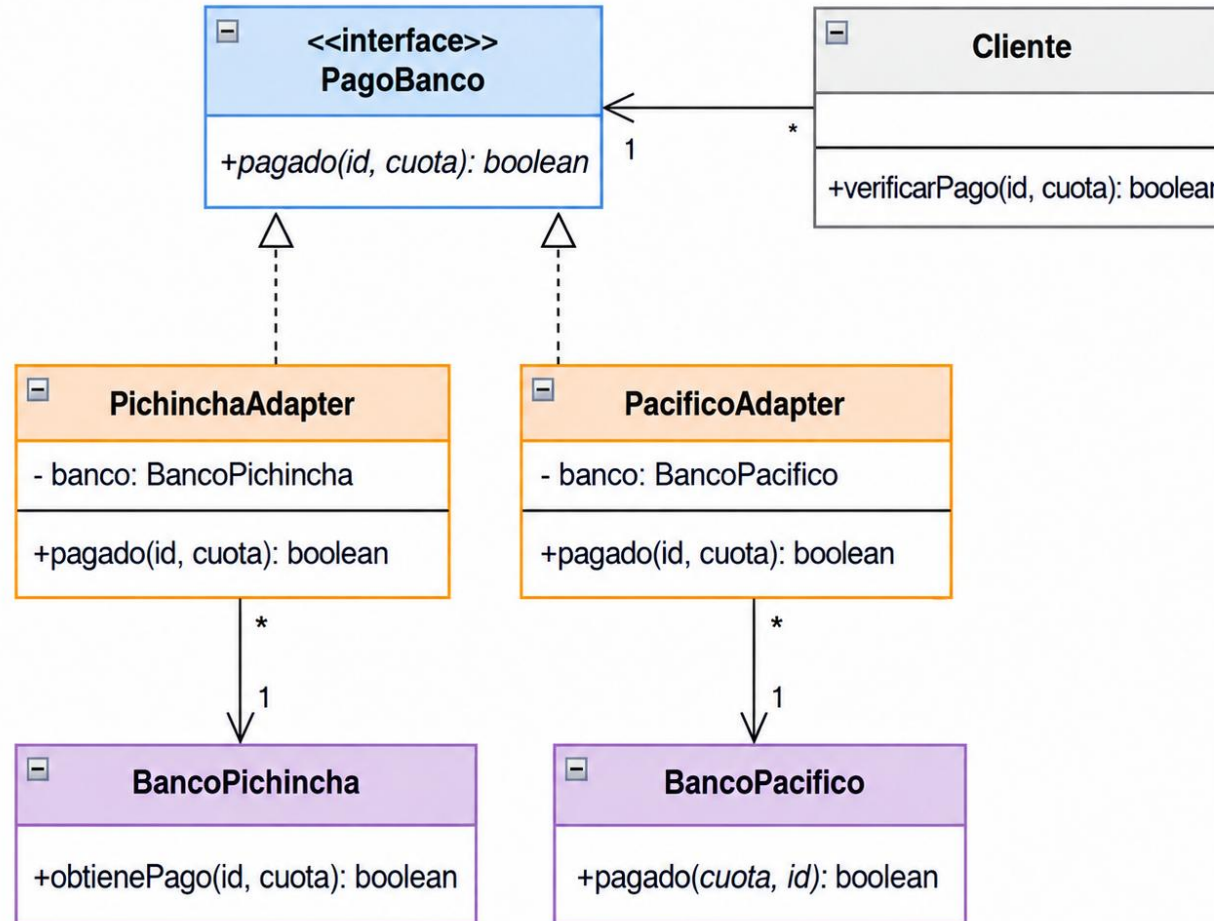
Mantiene una referencia al objeto adaptado cuando se utiliza un adaptador de objetos.

- Colaboraciones

- El Cliente trabaja únicamente con la interfaz Target.
- El Cliente envía una solicitud al Adapter mediante esa interfaz.
- El Adapter transforma o delega la solicitud hacia la operación correspondiente del Adaptee.
- El Cliente no necesita conocer la interfaz original del Adaptee.

Adapter

Solución



<https://github.com/mortizo/pds-unidad02-ooms/tree/master/pds-unidad02-ooms/src/main/java/p69/adapter>

Adapter

Adapter

- Aplicabilidad
 - Cuando necesitamos utilizar una clase o componente existente cuya interfaz no coincide con la que requiere nuestro sistema.
 - Cuando debemos integrar componentes externos, legados o de terceros sin modificar su implementación.
 - Cuando diferentes proveedores ofrecen funcionalidades equivalentes mediante interfaces distintas y queremos exponer un contrato común al cliente.
 - Cuando queremos aislar al dominio de cambios en APIs, librerías o sistemas externos.
- Evitarlo cuando: Podemos modificar directamente los componentes y acordar una interfaz común sin introducir una capa adicional de adaptación.
- Adapter es útil cuando el problema no está en lo que hace un componente, sino en cómo debemos comunicarnos con él.

Adapter

Actividad en clase: ¿Adapter?

Caso 1 — Autenticación

Una aplicación debe autenticarse mediante diferentes proveedores, por ejemplo OAuth, Active Directory y un servicio institucional propio.

Cada proveedor ofrece operaciones y estructuras de respuesta diferentes.

¿Cómo diseñarían una solución que permita autenticar usuarios mediante una interfaz común sin modificar los proveedores existentes?

Caso 2 — Proveedores de pago

Una tienda en línea integra diferentes proveedores de pago. Cada proveedor utiliza una API distinta para procesar transacciones y devolver sus resultados.

¿Cómo diseñarían la integración para que el sistema procese pagos mediante un mismo contrato, independientemente del proveedor?

Caso 3 — Servicios de correo

Una aplicación debe enviar correos utilizando diferentes servicios externos.

Cada servicio expone operaciones, parámetros y mecanismos de autenticación diferentes.

¿Cómo diseñarían el sistema para enviar correos mediante una interfaz uniforme sin acoplar el dominio a cada proveedor?

Caso 4 — Traducción externa

Una plataforma utiliza distintos servicios externos de traducción.

Cada proveedor recibe y devuelve la información mediante interfaces diferentes.

¿Cómo diseñarían la integración para que el sistema solicite traducciones mediante un contrato común?

Tema:

Facade

- Patrones estructurales
- Composite
- Adapter
- *Facade*

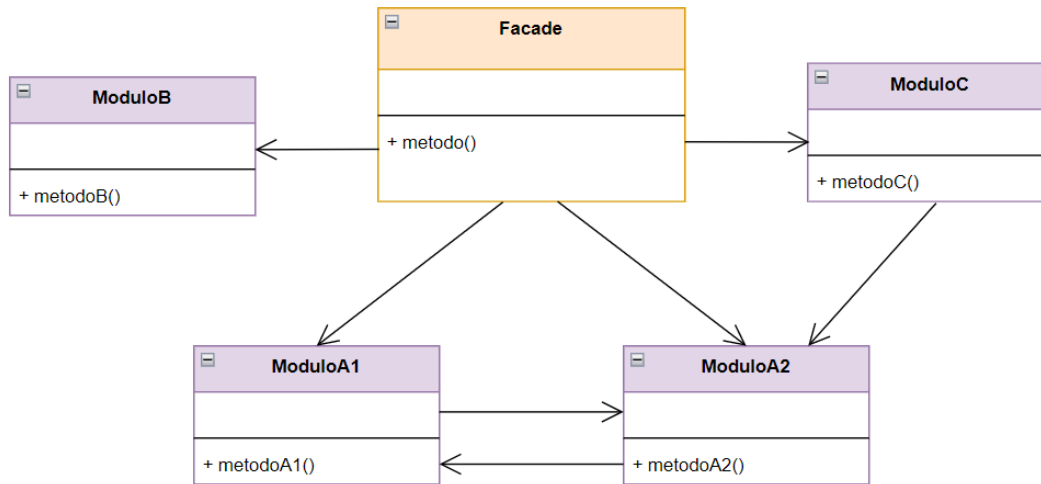
FACADE

FACADE

- Propósito
 - Proporciona una interfaz simplificada y unificada para acceder a un conjunto de clases o interfaces de un subsistema.
 - Define operaciones de alto nivel que permiten utilizar el subsistema sin que el cliente deba conocer todos sus componentes internos.
- Problema / Motivación
 - Una universidad mantiene la información de personas en módulos separados: Estudiantes, Docentes, Colaboradores y Proveedores.
 - Cada módulo ofrece sus propias operaciones y reglas de acceso.
 - Un cliente que necesita consultar información de una persona tendría que conocer qué módulo utilizar y cómo interactuar con cada uno.
 - Esto aumenta el acoplamiento entre el cliente y los detalles internos del subsistema.

FACADE

Estructura



• Participantes

• Fachada:

Proporciona operaciones de alto nivel para acceder al subsistema.

Conoce qué objetos del subsistema deben intervenir y coordina sus llamadas.

• Clases del subsistema:

Implementan la funcionalidad real del sistema.

Realizan las operaciones solicitadas por la Fachada.

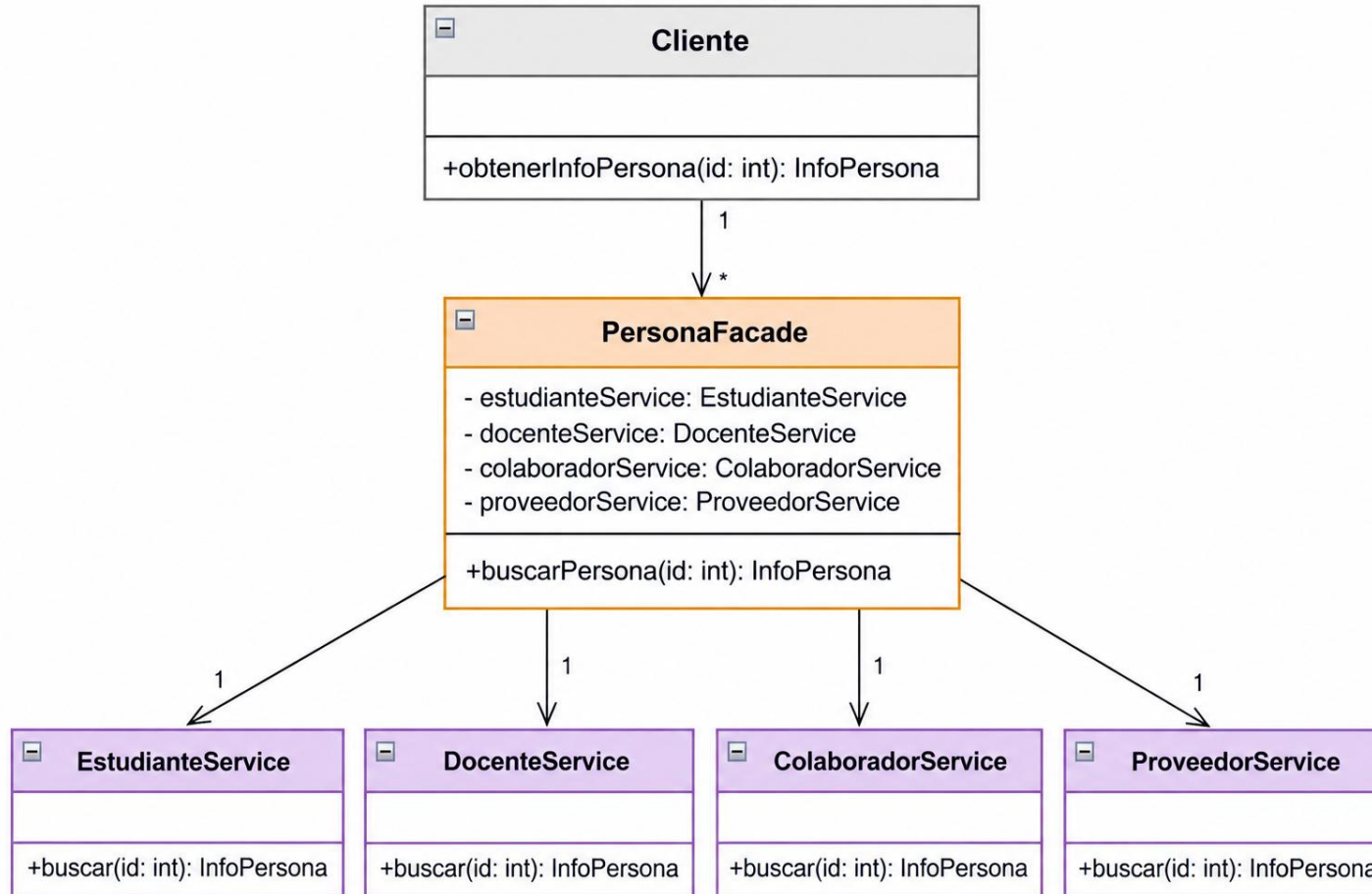
No necesitan conocer la existencia de la Fachada.

• Colaboraciones

- El Cliente realiza las operaciones habituales mediante la Fachada.
- La Fachada coordina y delega las solicitudes hacia los objetos apropiados del subsistema.
- Las clases del subsistema realizan el trabajo real.
- Cuando sea necesario, un cliente especializado puede seguir accediendo directamente a las interfaces del subsistema.

FACADE

Solución



<https://github.com/mortizo/pds-unidad02-ooms/tree/master/pds-unidad02-ooms/src/main/java/p69/facade>

FACADE

FACADE

- Aplicabilidad
 - Cuando un subsistema ofrece muchas clases o interfaces y queremos proporcionar a los clientes un punto de acceso más simple.
 - Cuando deseamos reducir el acoplamiento entre los clientes y los detalles internos de un subsistema.
 - Cuando necesitamos ofrecer operaciones de alto nivel que coordinan varias operaciones internas.
 - Cuando queremos establecer puntos de entrada claros entre capas o subsistemas.
- Evitarlo cuando:
 - La interfaz de la Fachada empieza a concentrar demasiadas responsabilidades o lógica de negocio.
 - El cliente necesita trabajar directamente con funcionalidades especializadas que la Fachada no debería ocultar.
- Facade reduce lo que el cliente necesita conocer del subsistema.

Facade

Actividad en clase: ¿Facade?

Caso 1 — Reserva de vuelos

Una plataforma de viajes debe coordinar autenticación, consulta de disponibilidad, selección de vuelo, procesamiento de pago y emisión de la reserva.

¿Cómo diseñarían un acceso simplificado para que el cliente pueda realizar una reserva sin conocer todos los componentes internos del sistema?

Caso 2 — Procesamiento de imágenes

Una aplicación permite cargar imágenes, recortarlas, redimensionarlas, aplicar filtros y guardar el resultado.

Cada operación es gestionada por componentes especializados.

¿Cómo diseñarían una interfaz de alto nivel que permita ejecutar estas operaciones sin que el cliente deba coordinar directamente todos los componentes internos?

Caso 3 — Gestión de pagos

Un sistema de pagos debe validar datos, autorizar la transacción, registrar el movimiento, generar el comprobante y notificar el resultado.

¿Cómo diseñarían un punto de acceso simplificado que coordine estas operaciones sin exponer al cliente toda la complejidad del subsistema?

Caso 4 — Matrícula universitaria

Para matricular a un estudiante, el sistema debe validar requisitos académicos, verificar obligaciones financieras, consultar cupos, registrar asignaturas y emitir la confirmación.

¿Cómo diseñarían una interfaz de alto nivel que permita ejecutar el proceso de matrícula sin que el cliente tenga que interactuar directamente con todos estos módulos?

Unidad 02.01

Retroalimentación y cierre

Qué vimos hoy

- Composite: permite tratar de manera uniforme objetos individuales y composiciones dentro de estructuras jerárquicas.
- Adapter: permite colaborar con componentes existentes cuya interfaz no coincide con la que necesita el cliente.
- Facade: proporciona un acceso simplificado a un subsistema sin exponer innecesariamente su complejidad interna.
- Los tres son patrones estructurales, pero resuelven problemas diferentes de organización y relación entre objetos.
- Aplicar un patrón estructural solo tiene sentido cuando existe un problema estructural real que justifique la abstracción adicional.

Antes de finalizar:

- ¿Qué diferencia fundamental existe entre Adapter y Facade?
- ¿Qué característica permite reconocer rápidamente un posible Composite?
- ¿Qué riesgo aparece si aplicamos cualquiera de estos patrones sin necesidad?

Idea final: Un patrón estructural no mejora un diseño por existir; aporta valor cuando organiza una relación problemática con menor acoplamiento y una responsabilidad claramente definida.

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Anexos

Links de Referencia

- <https://codideep.com/blogpost/stupid-vs-solid>
- <https://refactoring.guru/design-patterns>
- <https://reactiveprogramming.io/books/design-patterns/es>
- https://es.wikipedia.org/wiki/Barbara_Liskov
- <https://www.geeksforgeeks.org/composite-design-pattern/>

Gracias por su atención

posgrados.ups.edu.ec